THE NATIONAL UNIVERSITY OF LESOTHO

LIMO Simulator Design

Milestone M2 Submission · CS3520 Computer Organisation and Architecture · LIMO Group Project · AY 2026/2027

Team	limo-nul	**Release tag**	m2
Lab group	C	**Commit**	[short SHA of the m2 tag]
Due	Sun 11 Oct 2026, 23:59	**Live site**	https://recocipher-beep.github.io/limo-nul/
Repository	https://github.com/recocipher-beep/limo-nul	**Release**	https://github.com/recocipher-beep/limo-nul/releases/tag/m2

#	Member	Student no.	GitHub	M2 role
1	LEREKO THULO	202322270	recocipher-beep	Test lead
2	QABA MOIKETSI	202002573	qabamothe	Interface lead
3	TUMELO BERNARD LEPUTLA	202403700	TumeloLeputla2006	Pipeline and hazard-unit lead
4	SETENTE NTHAKO	202321761	Setente	Assembler lead
5	THATO LEPITLA	202201975	Thatosh	Scrum and release lead
6	TS'EBETSO JADEN LEKHANYA	202322765	jadenlekhanya	ISA architect
7	(no seventh member assigned)	-	-	-

Assessment (M2). Weekly compliance: on time, tagged, sprint note (part of the 8 marks for M2–M5). M2 contents required by the handout: ISA frozen with D1 feedback applied; architecture and block diagram; pipeline-register data model; interface wireframes; working assembler with unit tests; site live on GitHub Pages; tag m2.

1 Team charter update

Item	Agreement
Project board	https://github.com/recocipher-beep/limo-nul (GitHub Projects board, one issue per deliverable, every issue has an owner)
Meeting rhythm	Stand-up Fri and Sun 17:00–21:00 (unchanged from M1)
Decision rule	Majority rules (unchanged); the role owner has the final word on technical detail inside their own role
Definition of done	Merged through a reviewed pull request, `node tests/run_tests.js` passes, issue closed
Branch and PR rules	`main` is protected. Work happens on a branch named `m2/<role>-<topic>`; a pull request needs one approving review from a different member; every member commits from their own account
Absent / not contributing	Verbal notice, then contact; persistent absence is reported to the coordinator through the peer assessment (unchanged from M1)

Role (M2)	Owner	Main deliverable	Files	PR reviewer
ISA architect	Ts'ebetso	ISA v1.0 freeze, errata, memory map	src/isa.js	Setente
Assembler lead	Setente	Two-pass assembler, error messages	src/assembler.js	Ts'ebetso
Test lead	Thulo	Unit tests, 8 test programs, reference model	tests/, src/examples.js, examples/	Leputla
Pipeline and hazard-unit lead	Leputla	Architecture, data model, control table, hazard rules	src/pipeline_model.js	Thulo
Interface lead	Moiketsi	Editor/assembler interface, wireframes	index.html, src/ui.js, src/style.css	Thato
Scrum and release lead	Thato	Board, README, AI_USE, Pages, tag m2	README.md, AI_USE.md, LICENSE, package.json, docs/	Moiketsi

2 ISA freeze

The LIMO ISA is **frozen as v1.0**: the same 20 instructions, formats, opcodes, funct values, register numbers and Sesotho names as M1. No encoding changed; the five hand-encoded words of M1 section 2.5 are reproduced bit for bit by the assembler (tests A1–A5). Corrections and clarifications found while building M2:

#	Change since M1	Why	Affects encodings?
C1	Estimated cycles corrected: Program 1 = **64** (was 66), Program 2 = **53** (was 55). Design-decision (c) rewritten.	The last taken transfer has no following instruction, so its two flush cycles never delay anything: cycles = N + 4 + stalls + 2 × (taken transfers followed by another instruction). Program 3 (16) was already right.	No
C2	Memory size filled in: 4 KiB instruction memory + 4 KiB data memory, separate; all registers and memory are zero at reset.	M1 left "[memory size]" empty. 4 KiB fits the 12-bit offsets and the 1024-instruction limit.	No
C3	Hazard unit compares only the source registers an instruction really reads (rs1/rs2 flags in the control table). Design-decision (d) reworded.	In I-type instructions the bits at the rs2 position are immediate bits; comparing them would cause false stalls.	No
C4	Register file writes in the first half of a cycle and ID reads in the second half (write-before-read). R0 is never forwarded.	Removes the need for a WB→ID forwarding path; a result is visible 3 instructions later through the register file.	No
C5	Assembler conventions (v1.1): hex (0x1F) and binary (0b101) immediates, register aliases (sp, a0, khang1 ...), branch/jump distances must be multiples of 4, apostrophes allowed inside labels.	Handout B1 requires decimal and hex immediates; offsets of 2 can be encoded but would point into the middle of an instruction.	No
C6	Team name and tag placeholders in the page footer replaced (limo-nul, tag m2).	Housekeeping from M1.	No

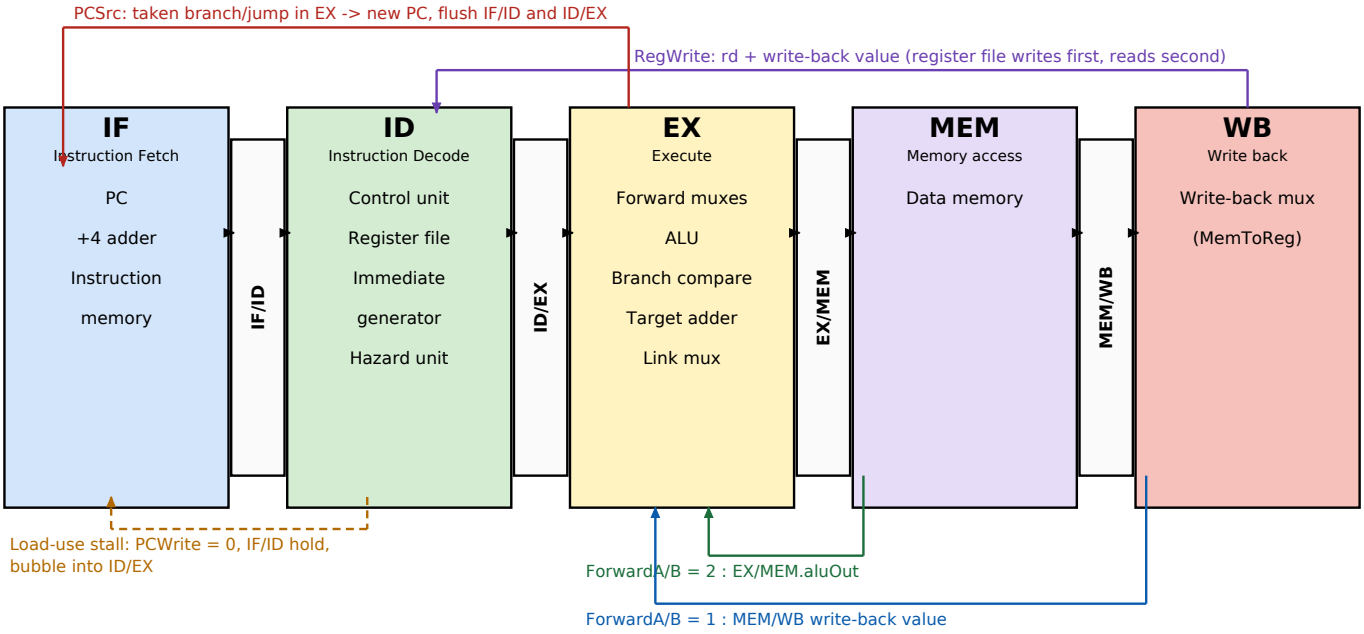
D1 feedback. [TO BE COMPLETED BY THE TEAM BEFORE SUBMISSION: copy each comment the examiner gave at Defence 1 and write beside it what was changed, or why nothing needed changing. The ISA architect owns this table.]

Design-decision log corrections (replace the M1 text; each is under 120 words, verified by script).

(c) Branch resolution (118 words). LIMO resolves branches and jumps in EX and predicts not taken. A taken transfer flushes the two younger instructions in IF and ID: a two-cycle penalty. Program 1 has ten taken transfers (nine tl, final khk), but the last delays nothing: no instruction follows it. The run takes 42 instructions + 4 fill cycles + 9 x 2 flush cycles = 64 cycles. Resolving in ID would cut each flush to one cycle, but needs comparison hardware, forwarding into ID, and a stall when a branch follows the instruction producing its operand. Here khk follows kpnh R5 every pass, so ID resolution takes 42 + 4 + 9 + 10 stalls = 65 cycles. EX reuses existing hardware.
(d) Load-use (112 words). Forwarding can send a result from a pipeline register to an ALU input, but a load's data is not available until the end of MEM. In Program 3, tsl R6, 0(R11) reads memory in cycle 4 of the diagram below, while the next instruction, kpn R7, R5, R6, would need R6 at the start of its EX stage in that same cycle. The processor inserts one stall and forwards the loaded value from MEM/WB to EX in cycle 5. Program 3 therefore takes 16 cycles for 11 instructions. The hazard unit stalls when ID/EX.MemRead is set and ID/EX.rd equals a source register (rs1 or rs2) that the instruction in ID really reads.
Sample programs (corrected estimates). Program 1: R6 = 55, 42 instructions, **64** cycles. Program 2: R5 = 0, 31 instructions, **53** cycles. Program 3: 11 instructions, **16** cycles (one load-use stall). Rule used: cycles = N + 4 + load-use stalls + 2 × (taken transfers followed by another executed instruction).

3 Architecture and block diagram

LIMO is a classic five-stage pipeline (IF, ID, EX, MEM, WB) with separate instruction and data memories, four pipeline registers, a forwarding unit, a load-use hazard unit, and branch/jump resolution in EX with a predict-not-taken policy.



Separate instruction memory (IF) and data memory (MEM). Solid arrows: data; dashed: stall control.

Figure 1: LIMO five-stage pipeline. Forwarding paths (green, blue) feed the EX stage; a taken branch or jump redirects the PC from EX (red); a load-use stall freezes PC and IF/ID (orange, dashed).

Stage	What happens	Units
IF	Fetch the word at PC from instruction memory; PC ← PC + 4 unless redirected or stalled.	PC, +4 adder, instruction memory
ID	Decode; read rs1 and rs2 from the register file; build the sign-extended immediate; generate control signals; detect a load-use hazard.	Control unit, register file, immediate generator, hazard unit
EX	Select forwarded operands (ForwardA/B); ALU operation or address calculation; compare for branches; compute branch/jump target; choose PC+4 as result for jumps; redirect PC and flush if taken.	Forward muxes, ALU, branch comparator, target adder, link mux
MEM	tsl reads, bl writes the data memory (word-aligned, little-endian).	Data memory
WB	Write aluOut or memData (MemToReg) to rd; writes to R0 are ignored.	Write-back mux, register file

4 Pipeline-register data model

Register	Fields held (in `src/pipeline_model.js`)	Written by	Read by
IF/ID	valid, pc, instr (32 b); pc4 = pc + 4 (derived, kept for display)	IF	ID
ID/EX	valid, pc, instr, rs1, rs2, rd (5 b each), rs1Val, rs2Val, imm (32 b, sign-extended), f3, and all control signals: regWrite aluSrc aluCtrl memRead memWrite memToReg branch jump jumpReg useRs1 useRs2	ID	EX, hazard unit
EX/MEM	valid, pc, instr, rd, aluOut (32 b: ALU result, or PC+4 for jumps), storeData (32 b, forwarded rs2), taken, target, control: regWrite memRead memWrite memToReg	EX	MEM, forwarding unit
MEM/WB	valid, pc, instr, rd, aluOut, memData (32 b), control: regWrite memToReg	MEM	WB, forwarding unit
Architectural state	pc; regs[32] (R0 always 0); imem (instruction words); dmem (4096 bytes, little-endian); cycle; stats {instructions, stalls, flushes}	-	-

The same field names are used in the JavaScript object `createCpuState()` (src/pipeline_model.js), in the Datapath view, and in the tests, so the interface can show every pipeline register directly. A bubble is a register with `valid = false` and all control signals 0.

Control signals generated in ID

Class (instructions)	RegW	ALUSrc	ALUCtl	MemR	MemW	M2R	Br	Jmp	JReg	rs1	rs2
R-type (kpn tls le kp kpe stts stto)	1	0	by funct	0	0	0	0	0	0	1	1
I-type ALU (kpnh leh kph kpeh)	1	1	by funct3	0	0	0	0	0	0	1	0
Load (tsl)	1	1	ADD	1	0	1	0	0	0	1	0
Store (bl)	0	1	ADD	0	1	0	0	0	0	1	1
Branch (khl khsl khn khk)	0	0	-	0	0	0	1	0	0	1	1
Jump and link (tlk, tl)	1	0	-	0	0	0	0	1	0	0	0
Jump register (tlkr)	1	1	ADD	0	0	0	0	1	1	1	0

Key: RegW = RegWrite, ALUCtl = ALUCtrl, MemR = MemRead, MemW = MemWrite, M2R = MemToReg, Br = Branch, Jmp = Jump, JReg = JumpReg; rs1 / rs2 = 1 when the instruction really reads that source register (used by the hazard unit). "-" = not used. Hazard rules. **Forwarding:** ForwardA/B = 2 if EX/MEM.regWrite and EX/MEM.rd ≠ 0 and EX/MEM.rd equals the source; else 1 if MEM/WB matches; else 0 (register file). **Load-use:** stall one cycle when ID/EX.MemRead = 1, ID/EX.rd ≠ 0 and rd equals a source register that the instruction in ID reads. **Control:** a taken branch/jump in EX flushes IF/ID and ID/EX (two bubbles).

5 Interface wireframes

The interface is a static web page (HTML/CSS/JavaScript) that works offline. M2 delivers the editor and assembler view (Figure 2, built and live); the pipeline, datapath, control, hazard and statistics views are wireframed (Figure 3) and appear as dashed tabs marked M3/M4.

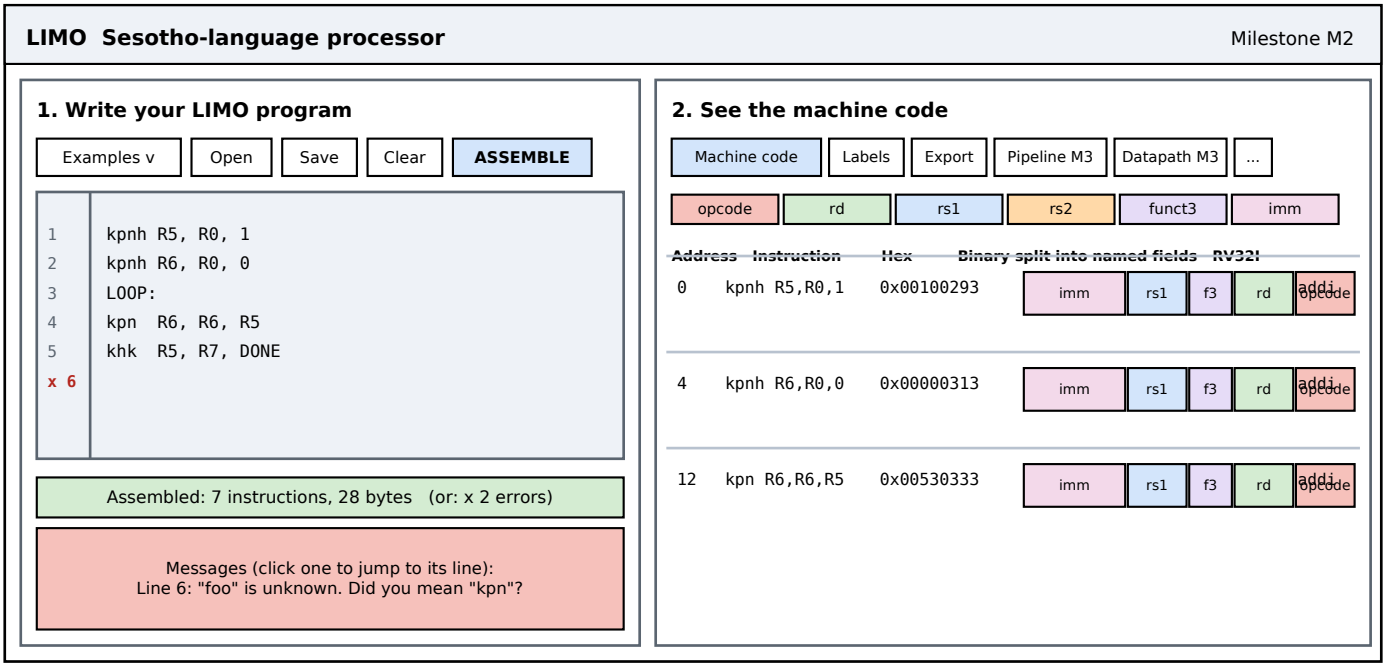


Figure 2: Editor and assembler view (implemented in M2). Colour is never the only cue: every field is named, error lines carry an "x" mark.

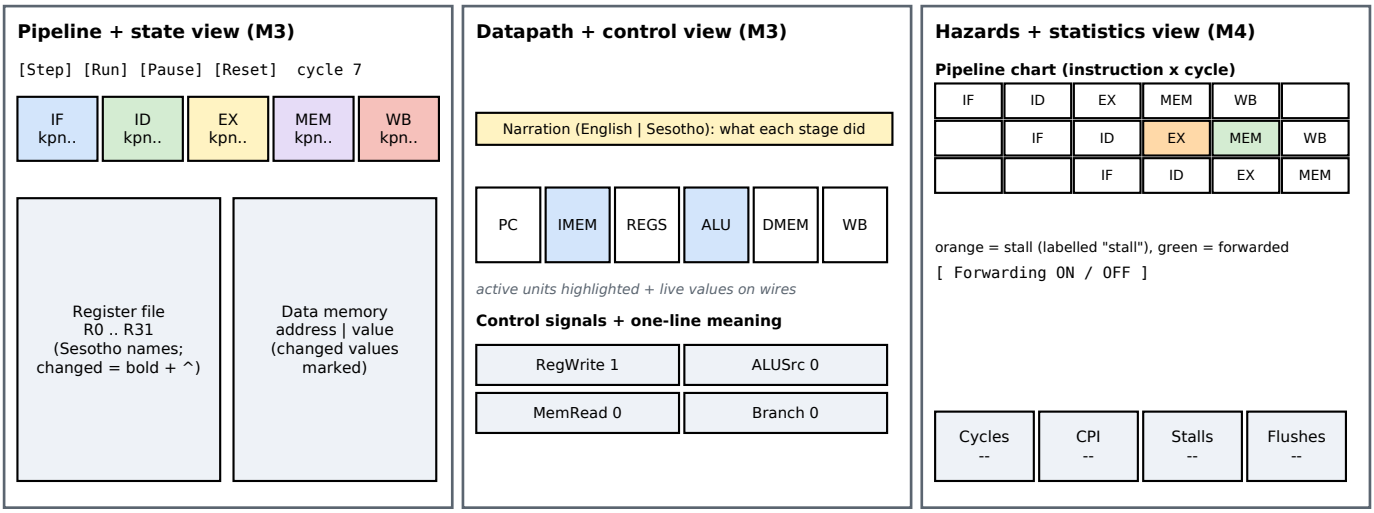


Figure 3: Wireframes of the M3/M4 views.

6 Assembler and unit tests

The assembler (src/assembler.js) is a two-pass assembler: pass 1 removes comments, records labels and counts instructions; pass 2 parses operands, checks ranges and encodes. It reports every error in plain language with its line number, accepts decimal/hex/binary immediates, register aliases and labels with apostrophes, and produces the RV32I translation used for the Ripes cross-check. Run all tests with one command: `node tests/run_tests.js` (needs only Node.js).

Test group	Passed	Failed
A. Assembler: encodings (hand-derived, same as RV32I)	25	0
B. Assembler: syntax features	12	0
C. Assembler: error messages with line numbers	16	0
D. Control decoder (ID stage)	8	0
E. Hazard unit: forwarding, load-use stall, branch comparator	15	0
F. The 8 LIMO test programs: results and cycle counts (hand-derived)	12	0
Total	**88**	**0**

The 8 test programs (handout B8-ii) with hand-derived registers, memory and cycle counts:

Program	Executed	Stalls	Taken	Cycles	Formula
1. Sum from 1 to 10 (loop)	42	0	10	**64**	42 + 4 + 0 + 2x9
2. Countdown from 10 to 0	31	0	10	**53**	31 + 4 + 0 + 2x9
3. Memory, load-use stall, mask and shift	11	1	0	**16**	11 + 4 + 1
4. RAW hazards resolved by forwarding	5	0	0	**9**	5 + 4
5. Logic, shifts and immediates	12	0	0	**16**	12 + 4
6. All four branches, taken and not taken	9	0	2	**17**	9 + 4 + 2x2
7. Array loop: store 5 numbers, add them up	39	5	4	**56**	39 + 4 + 5 + 2x4
8. Call and return (tlk / tlkr)	6	0	3	**14**	6 + 4 + 2x2

7 Publication and release

- Site: **https://recocipher-beep.github.io/limo-nul/** (GitHub Pages, branch main, folder /root; total size about 55 KB, limit 2 MB).
- Repository layout: /src, /tests, /examples, /docs (ISA specification, design, test report: the seven PDFs of this milestone), README.md, AI_USE.md, LICENSE.
- Release: tag **m2** → https://github.com/recocipher-beep/limo-nul/releases/tag/m2

8 References

- Patterson, D. A. and Hennessy, J. L. *Computer Organization and Design: The RISC-V Edition*, Chapters 2 and 4.
- Harris, S. L. and Harris, D. *Digital Design and Computer Architecture: RISC-V Edition*, Chapters 6–7.

- The RISC-V Instruction Set Manual, Volume I: Unprivileged ISA (RV32I).
- [Sesotho sources or speakers consulted for the mnemonics: TO BE COMPLETED BY THE TEAM (empty since M1)]

9 AI-use declaration

Tool	Used for	How we checked or changed the output
Claude (Anthropic)	Drafting the M2 design text, JavaScript code (assembler, pipeline model, interface), unit tests and PDFs; finding the cycle-count errata of M1	Ran `node tests/run_tests.js` (88 tests pass); opened the live site and assembled all 8 examples; compared the five M1 hand-encoded words. [Team: add here what each member edited or rewrote.]
ChatGPT	[as declared in M1; update if used again]	[update]

We confirm that every member has reviewed the submission and can explain the design, the code of their role, the encodings and the sample programs.

Sprint note (maximum one page; not counted in the page limit)

Sprint goal: Freeze the ISA, design the simulator, deliver a working assembler with unit tests and a live interface, and publish tag m2.

Member	Role this sprint	Delivered (issue and PR numbers)	PRs reviewed
LEREKO THULO	Test lead	tests/run_tests.js (88 tests), tests/ref_model.js, 8 test programs, test plan PDF — issue #__ / PR #__	PR #__ (pipeline)
QABA MOIKETSI	Interface lead	index.html, src/ui.js, src/style.css, wireframes, interface PDF — issue #__ / PR #__	PR #__ (admin)
TUMELO BERNARD LEPUTLA	Pipeline and hazard-unit lead	src/pipeline_model.js, block diagram, data model, control table, PDF — issue #__ / PR #__	PR #__ (tests)
SETENTE NTHAKO	Assembler lead	src/assembler.js, error catalogue, assembler PDF — issue #__ / PR #__	PR #__ (ISA)
THATO LEPITLA	Scrum and release lead	README, AI_USE, LICENSE, package.json, board, Pages, tag m2, sprint PDF — issue #__ / PR #__	PR #__ (interface)
TS'EBETSO JADEN LEKHANYA	ISA architect	src/isa.js, ISA v1.0 freeze, errata, memory map, PDF — issue #__ / PR #__	PR #__ (assembler)

Done	Everything listed in the handout for M2 (see sections 2–7).
Not done, and why	M3 and M4 content (pipeline execution, visual hazards): scheduled for later sprints. D1 feedback table and Sesotho sources: to be filled by the team.
Next sprint	M3: pipeline core (five stages running, Step/Run/Reset, register and memory views).
Blockers	[Internet connection / other: update]

Replace every "#__" with the real issue/PR number from the GitHub board before submitting.